# (Clustering) Assignment 4

Shawn Jiang

Dr Elodie Lugez

## Introduction

My data is from the UCI machine learning repository and it is essentially a table of 210 seed entities. From left to right, the features displayed are as follows: area, perimeter, compactness, length of kernel, width of kernel, asymmetry coefficient, and length of kernel groove. All of the parameters are real-valued and continuous. The seeds (kernels) belong to 3 different varieties of wheat: Kama, Rosa, and Canadian, and there are 70 elements of each for a total of 210 table entries. Obviously, because this is a clustering assignment, we do not care for the class label in this context.

Based on the above description, our goal is to employ a clustering algorithm to divide all the kernels into appropriate clusters in which the elements of each cluster all match one type of kernel. Thus, we expect to have 3 clusters ultimately.

I find this dataset to be interesting for a few reasons. For one, this is a real-world agricultural dataset used in wheat breeding and grain quality classification. The features in particular, relate to shape, symmetry, and size, which are all meaningful in real biological classification. In addition, the dataset contains measurements extracted from X-ray images of wheat kernels, making it an early example of a real industrial computer-vision dataset.

## Concepts

In this section, I describe important concepts that will be crucial for understanding the rest of the sections of this assignment. For the approach, we will be employing the 2 partitional clustering algorithms of bisecting K-means and DBSCAN, and thus we will primarily be discussing these 2 algorithms. Partitional clustering indicates that our goal will be to partition the dataset into distinct clusters that are disjoint from each other and in which the union of all the clusters forms the original dataset.

### Bisecting K-means

Before explaining bisecting K-means, we must discuss what the K-means algorithm is. Basically, within the context of K-means, the definition of a cluster is a group of data points in which each data point of the cluster is closer (assessed via Euclidean distance) to the centroid of the cluster than to the centroid of any other cluster.

In K-means clustering, we start with the dataset and then we set the hyperparameter of how many clusters we want to have. Suppose we want K clusters. Then we randomly initialize the centroids of the K clusters on the plane containing the dataset and we assign all the points to a centroid based on whichever one they are closest to. From this, we basically have an initial clustering which is very likely to be far from perfect. As a result, we gradually move closer to the

ideal clustering (for this initialization of K clusters) by recomputing the centroids of these initial clusters and reassigning the points once more. We do this until the centroids barely change after each iteration, signifying that the clusters have converged. The problem with employing this approach is that the assignment of the initial centroids may have a big impact on the final clustering that results. This is where bisecting K-means comes in.

In bisecting K-means clustering, we address the aforementioned issue by doing many iterations of the choosing initial centroids part and then use SSE (Sum of Squared Error) to determine which set of clusters from these iterations is the most optimal one. Essentially, SSE is a quantity computed by adding up the squared distances between all the points of the clustering and the centroids that the points are associated with. The formula is shown below in Figure 1:

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist^2(m_i, x)$$

Figure 1

K = total number of clusters in our clustering
C_i = ith cluster
x = point in ith cluster
dist^2(x, y) = distance between points x and y squared
m_i = centroid of C_i

Note that for K-means, as the clusters converge, the SSE also converges to a global minimum. In general, a lower SSE is more indicative of the fact that the clusters derived from our algorithm are representative of genuine separate groups in a real-life setting and provide validation for our ascertained clusters.

1: Initialize the list of clusters to contain the cluster containing all points.
2: **repeat**
3: Select a cluster from the list of clusters
4: **for** $i = 1$ to $number\_of\_iterations$ **do**
5: Bisect the selected cluster using basic K-means
6: **end for**
7: Add the two clusters from the bisection with the lowest SSE to the list of clusters.
8: **until** Until the list of clusters contains $K$ clusters

Figure 2

Figure 2 showcases the bisecting K-means clustering algorithm. We can see that on a high level, we are gradually bisecting the dataset, introducing 2 new clusters each time until we reach K clusters. The key thing is that during each bisecting stage, we choose an existing cluster from our list of clusters (that we deem needs to be split up based on SSE or some other measure) and instead of just using K-means to partition this current cluster into two clusters, we run many iterations where each time we choose a different set of random centroids to work with. This allows us to choose the optimal bisection by computing the SSE of all the candidates and choosing the bisection with the lowest one.

DBSCAN

A K-means approach does have some drawbacks, as it makes some assumptions about the data to produce the most optimal clustering. For one, it doesn't take density into account, as in, we can imagine if we have an incredibly dense globular cluster that is relatively close to the edge of a more sparse but still well-defined globular cluster (with relatively the same elements). From a visual standpoint, we as humans can easily identify the 2 separate clusters, but based on K-means, it is extremely common for the more dense cluster to "steal" datapoints from the sparse cluster, as the outermost shell of the more sparse cluster may actually be closer to the centroid of the denser cluster than its own centroid. Thus, we introduce another algorithm called DBSCAN that is entirely density-focused. In density-based clustering, clusters are defined to be regions of high density separated from one another by regions of discernible low density.

In DBSCAN, we form clusters by associating core points with each other and assigning border points to the closest cluster and eliminating noise points. A core point is defined on a high level to be a point in regions of high density and the more technical definition is that it is a point with at least a pre-designated minimum number of points within a certain distance from it. Epsilon is used to represent this distance, and MinPoints is the minimum number of points and these are both hyperparameters.

The way the algorithm works is that we first label all the points as a core, border, or noise points. But in order to do this, we must find adequate values for both MinPoints and Epsilon.

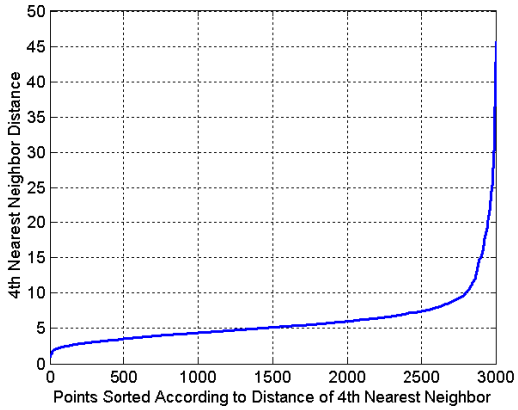



Figure 3

Figure 3 shows that we graph the distance to the 4th nearest neighbour vs number of points that have such a distance to the 4th nearest neighbour. We notice that if we set MinPoints = 4, then for each distance, we are essentially given the number of points that would be considered core points if we set Epsilon equal to that distance. For example, at Epsilon = 5, we see that there are 1500 points that are within a distance of 5 to their 4th nearest neighbour, indicating that all these 1500 points are core points since if they are at a distance of 5 to their 4th nearest neighbour, then there are exactly 4 points that are within an Epsilon distance from it. In addition, all the points at a distance below 5 to their 4th nearest neighbour are core points as well because they have exactly 4 points that are within a distance even smaller than 5 to them. Now notice between 2.5k and 3k points, there is a point where the rate of change climbs incredibly fast. At this shoulder, it is an indicator that the points beyond this point are incredibly far from their nearest neighbours and thus we are entering the noise point territory. Thus, we always choose the shoulder point to designate where we stop considering points to be core points. In this case, if we do assume MinPoints = 4, then the shoulder is somewhere at around Epsilon = 10 and thus any point within Epsilon distance to their 4th nearest neighbour would be considered a core point, which is all points within 10 or less to their 4th nearest neighbour. We can also do this for several k values and choose the graph with the steepest climb, which will be how we can choose MinPoints as well.

The rest of the algorithm to DBSCAN is incredibly simple. After labelling the core, border, and noise points, we connect core points with an edge where the points are within an Epsilon distance to each other, and we designate each connected graph as a distinct cluster. And finally, we assign each border point to one of the clusters.

**Methodology**

The idea is that we don't know the exact shape of the resultant optimal clusters, and thus, we choose to employ the above algorithms and see which one works better on this particular dataset.

First, we try DBSCAN. What we do is that we generate a chart, such as in figure 3, but this time, we generate a separate graph for various values of k (MinPoints), and we basically want to choose the graph with the steepest climb and choose the shoulder point of this graph for our Epsilon value as well. From our program, the following graph was produced.

(Shown on next page)

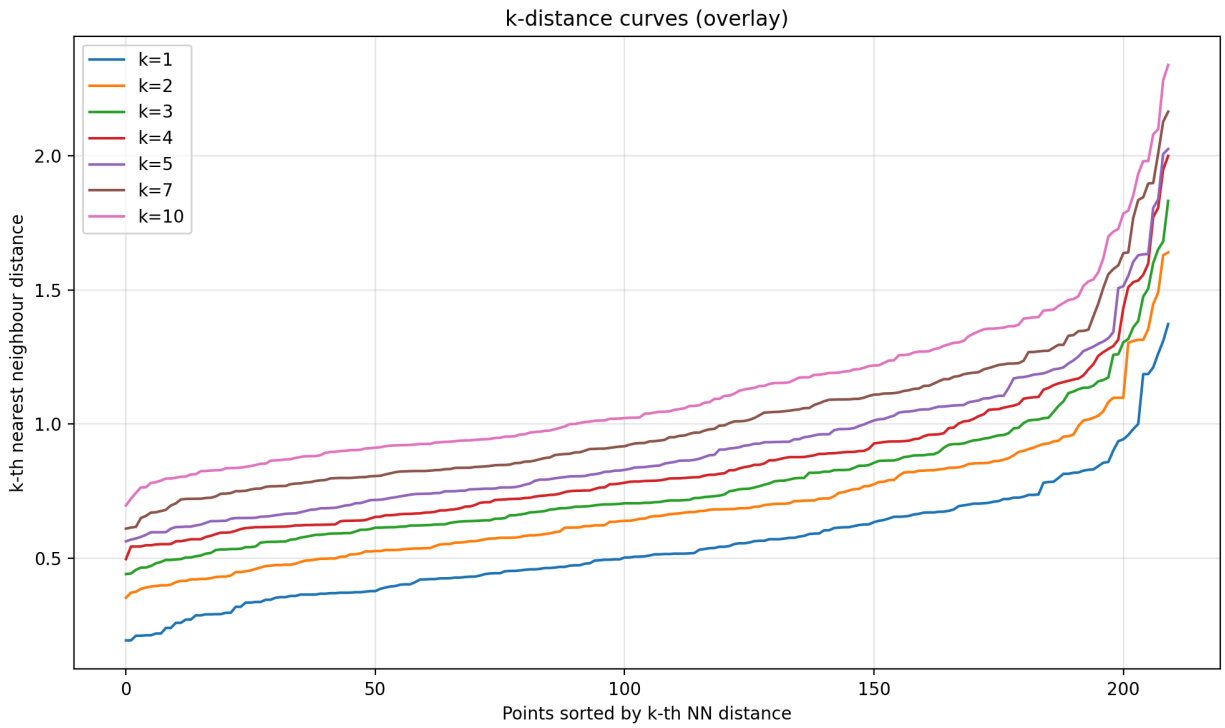



Figure 4

There were several graphs that looked workable here, so in the script, I decided to try 3 k values along with the associated Epsilon values. In particular, what I tried were:

1. MinPoints = 2, Epsilon = 1.1
2. MinPoints = 3, Epsilon = 1.12
3. MinPoints = 7, Epsilon = 1.325

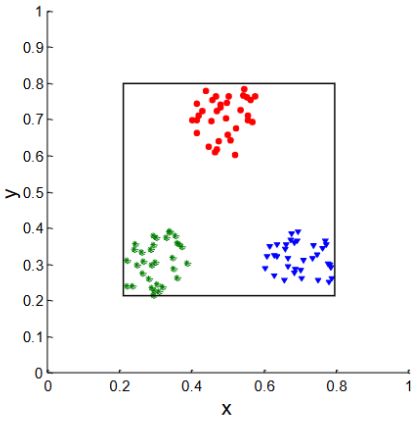
To evaluate, our script will output the number of clusters generated for each set of hyperparameters that we use, and we also display the total SSE for each run as well.

Next, we are going to try the bisecting K-means algorithm by assuming that the total clusters that we want in our clustering is K = 3. However, after reaching our final clustering, we need a way of assessing how good our clustering is. I decided to employ both a supervised and an unsupervised approach for this.

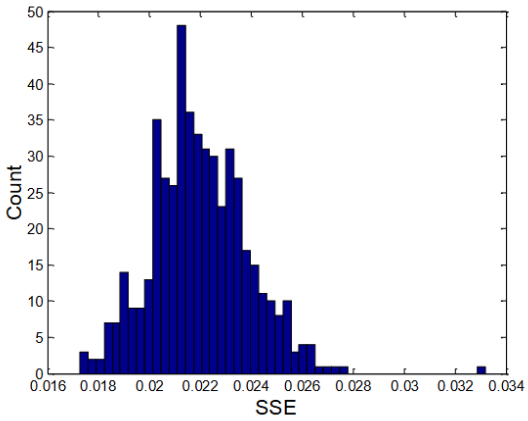
In the case of the supervised approach, this can be done since we are given the classes of all the kernels in our dataset and thus we can compute the percentage of “misclassifications” for our clusters by essentially identifying the ones in each cluster that deviate from the majority.

In the case of the unsupervised evaluation, we can use metrics such as SSE to assess the cohesion of our clustering. Our goal is increased cohesion, and thus we want the SSE to be as low as possible. However, an SSE value on its own has no meaning. We want it to indicate that

we have found some structure in our data that will provide validation for our clustering. In that case, this means that the null hypothesis we're working with is that the dataset is actually completely random, and we want to be able to show that the SSE value we have determined from our clustering, is incredibly rare and low under this null hypothesis assumption, thus indicating that it is likely that we have found a true structure in our data.




SSE = 0.005




Histogram shows SSE of three clusters in 500 sets of random data points of size 100 distributed over the range 0.2 – 0.8 for x and y values

Figure 5

We can see in Figure 5 that 500 sets of random data points of the size of the original dataset have been generated and an SSE calculated for each set. Plotting on a histogram, we notice that we get something that approximates a normal distribution, which is what we would expect in all cases that we do something similar to this. Next, we use this histogram to compute p-values, where we compute the probability (based on the generated histogram) that from a randomly generated dataset, we get an SSE value of equal to or lower than the actual SSE that we got from our original dataset. This probability can be represented as P(random_SSE <= actual_SSE), and if it is computed to be incredibly low, this gives us justification that our actual SSE is significant and that we have found a real structure in our data where we can meaningfully say that discernible clusters exist.

**Results**

After first trying the DBSCAN algorithm on the pre-designated hyperparameters, my script outputted some stats about the resultant clustering for each try or set of hyperparameters. Below are screenshots of the results.

```
DBSCAN params: eps = 1.1, min_samples = 2
  clusters found (excluding noise): 2
  noise points: 6
  within-cluster SSE (scaled space): 1385.438478
  label counts (label:count)  -- label -1 = noise:
    -1 : 6
    0 : 202
    1 : 2
```

```
DBSCAN params: eps = 1.12, min_samples = 3
  clusters found (excluding noise): 1
  noise points: 8
  within-cluster SSE (scaled space): 1385.316365
  label counts (label:count)  -- label -1 = noise:
    -1 : 8
    0 : 202
```

```
DBSCAN params: eps = 1.325, min_samples = 7
  clusters found (excluding noise): 1
  noise points: 5
  within-cluster SSE (scaled space): 1413.290403
  label counts (label:count)  -- label -1 = noise:
    -1 : 5
    0 : 205
```

We can see that in all these cases, we get something that is incredibly unfavourable and doesn't describe our dataset well given that we know what the class labels of each data point are. We know that the true number of clusters should be 3 but in all cases, the number of clusters that we get is below 2. This indicates to us that using DBSCAN was a bit of a mistake and that we should immediately move on to bisecting K-means clustering.

From bisecting K-means clustering with the hyperparameter of K set to 3, we computed an SSE value of 446.8687. Comparing this to the resultant SSEs from DBSCAN and we can already meaningfully say that an SSE of 446.8687 is a good sign. Next, we calculate p-values using exactly the same methodology described in **Methodology**. Our script calculated the p-value to be P(random_SSE <= actual_SSE) = 0. This means that given a random dataset, there is approximately a 0% chance that the SSE of the resultant clustering formed from this random dataset would be lower than our actual value from our real dataset. Thus, it is practically impossible for there to not be structure in our data and serves as greater evidence that our derived clustering serves as a relatively accurate way of describing the data.

```
Real Data Bisecting K-Means (k=3) SSE: 446.8687
Running 300 iterations of randomization test...
Mean Random SSE: 1608.4062
Std Random SSE: 41.6081
P-value (Prob Random SSE <= Real SSE): 0.000000
```

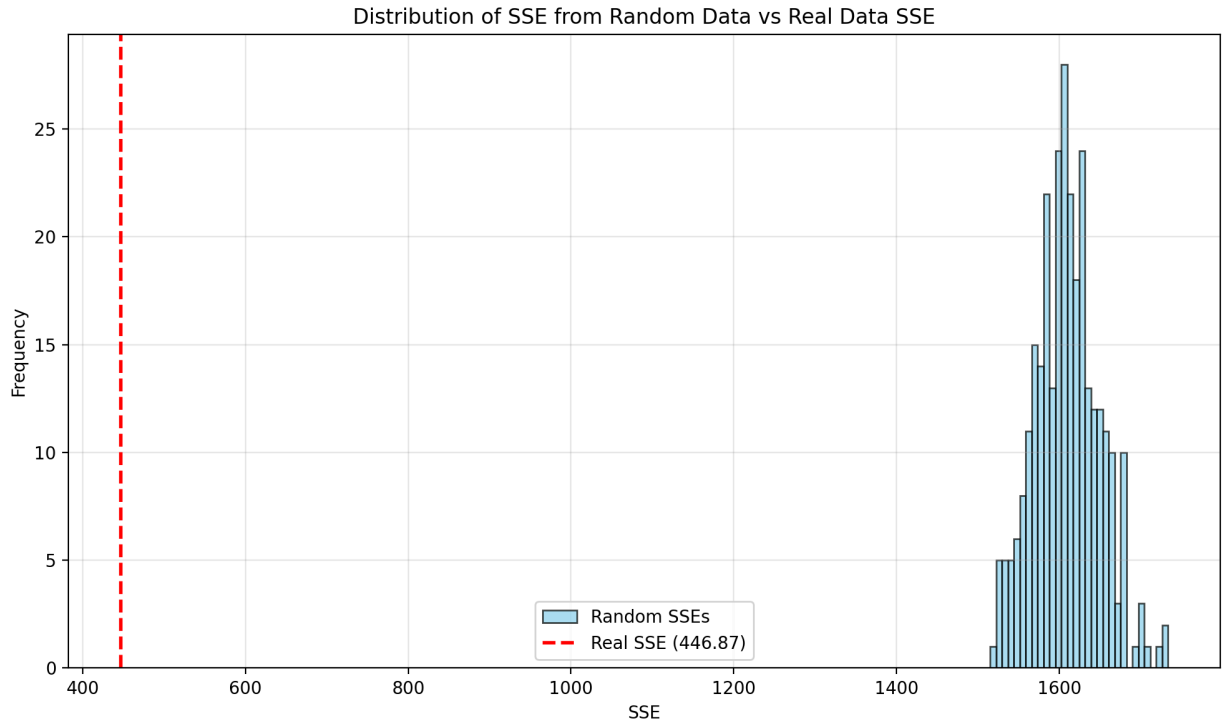



Figure 6

Our script also creates 3 new text files called cluster1.txt - cluster3.txt in which each text file stores all the data points corresponding to a particular cluster.

Cluster 1: Saved to cluster1.txt
  Total items: 77
  Majority Class: 2
  Misclassified items: 9
  Misclassification Percentage: 11.69%
---
Cluster 2: Saved to cluster2.txt
  Total items: 70
  Majority Class: 1
  Misclassified items: 10
  Misclassification Percentage: 14.29%
---
Cluster 3: Saved to cluster3.txt

Total items: 63
Majority Class: 3
Misclassified items: 1
Misclassification Percentage: 1.59%
---
We notice that for each cluster, we have a distinct majority class that covers the entire set of kernel types. This is a very good sign and further indicates that our clustering is a relative success. Not to mention, the misclassification percentages are also all relatively low, with the highest being 14.29% (still a little bit on the unfavourable side).

**Conclusion**

From the results, I can conclude that the DBSCAN algorithm was a futile attempt at clustering the dataset. This signifies that the dataset may vary in density throughout the plane in which the data points lie, rather than having relatively distinct and discrete density values where we have relatively clear boundaries between low and high density regions.

In the case of bisecting K-means, the clustering that we got from that was more or less successful. Our calculated p-value indicated to us that the SSE of the clustering was in a region in which we can confidently say that there is a structure to the data that we picked up on, and through a supervised approach of evaluation, we also concluded that the resultant clustering was relatively successful in partitioning the 3 kernel types into distinct clusters with some misclassification errors.

## References

Charytanowicz, M., Niewczas, J., Kulczycki, P., Kowalski, P., & Lukasik, S. (2010). Seeds [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5H30K.